Vaan Megam Networks

5G | Security | Private Network | Lab As A Service

TECHNICAL ASSIGNMENT SUBMISSION

Wireless Protocol Development Internship

TDMA Schedule Planner and Optimizer

Distance-2 graph colouring for collision-free TDMA scheduling, with schedule enforcement in the EMANE network emulator

---

Prepared by	Ramachandra TejSrikar Reddy
Institution	SRM Institute of Science and Technology B.Tech, graduating 2027
Submitted to	Vaan Megam Networks Private Limited IITM Research Park, Chennai
Scope	Part 1: Python Schedule Optimizer (mandatory) Part 2: EMANE emulation and integration bridge
Repository	github.com/srikarreddyram/tdma-schedule-optimizer
Date	5 October 2026

Abstract

This document describes a centralised TDMA schedule planner for a network of static radios and its integration with the EMANE network emulator. Part 1 is a Python command-line tool that builds the radio graph from node coordinates, derives the two-hop conflict graph and colours it with two greedy heuristics, DSATUR and largest-first, so that each colour becomes one time slot of a repeating frame. For the 16-node test network (a 4 × 4 grid with 300 m spacing and a 500 m range) the tool produces a **collision-free frame of 9 slots**, and a lower bound proves that no valid schedule for this layout can be shorter. Part 2 runs 16 EMANE TDMA radios in one Docker container and loads the Part 1 schedule into them through an integration bridge. With the optimizer's schedule EMANE recorded **no interference (SINR) losses in any run**, and with 10 ms slots 99.9% of packets were delivered across all in-range links, against 34.5% for a control schedule that separates only direct neighbours. With the 1 ms slots specified in the task, delivery ranged from 83.4% to 99.4% over three runs; that loss came from emulator timing, not from collisions.

Keywords: TDMA scheduling, distance-2 graph colouring, DSATUR, hidden terminal, spatial reuse, EMANE

Contents

1. Introduction 3
2. Requirements Traceability 3
3. System Model and Assumptions 4
4. Part 1: Design Process 5
5. Part 1: Final Values 7
6. Part 2: Design Process 9
7. Part 2: Final Values 11
8. Limitations and Future Work 13
9. Conclusion 13
Appendix A. Reproducing the Results 13
References 14

1. Introduction

1.1 Background

In a Time Division Multiple Access (TDMA) network all radios share one frequency and divide time into a repeating frame of fixed-length slots. Each radio transmits only in its own slots and listens in the rest, and a transmission is lost when its receiver hears a second transmission in the same slot. The schedule therefore has to keep interfering radios apart in time, a well-studied broadcast scheduling problem [5]. At the same time the frame should be short, because with one slot per radio per frame the frame length sets each radio's share of the channel and its access delay. Good schedules let distant radios transmit at the same time, which is known as **spatial reuse**, while keeping every receiver free of overlapping signals.

1.2 Scope and Key Results

The task asks for a centralised Python schedule optimizer (Part 1, mandatory) and its integration with a local EMANE emulation (Part 2, bonus). Both parts were completed. Section 2 maps each requirement to the place where it is met, Sections 3 to 5 cover the model, design and final values of Part 1, and Sections 6 and 7 do the same for Part 2. Table 1 summarises the results.

Table 1. Key results.

Measure	Result	Section
Frame length, 16-node grid	**9 slots**; DSATUR and largest-first agree	5.1
Lower bound for the grid	9 slots, so the schedule is **provably optimal**	4.5, 5.2
Spatial reuse on the grid	5 of the 9 slots shared; 12 of the 16 radios transmit in a shared slot	5.1
Collisions found by the validator	**0**	4.6
Automated tests	98 pytest tests	5.5
EMANE delivery, 10 ms slots	**99.9%** with the optimizer schedule; 34.5% with the distance-1 control	7.2
EMANE SINR drops, optimizer schedule	**0 bytes** in every run, at 1 ms and at 10 ms	7.1
EMANE delivery, 1 ms slots	83.4% to 99.4% over three runs; all loss due to slot timing	7.3

2. Requirements Traceability

Table 2 lists each requirement of the problem statement, how it is met, and where the evidence is. R1 to R9 belong to Part 1, R10 to R13 to Part 2, and R14 and R15 to the submission as a whole.

Table 2. Requirements of the problem statement and where they are addressed.

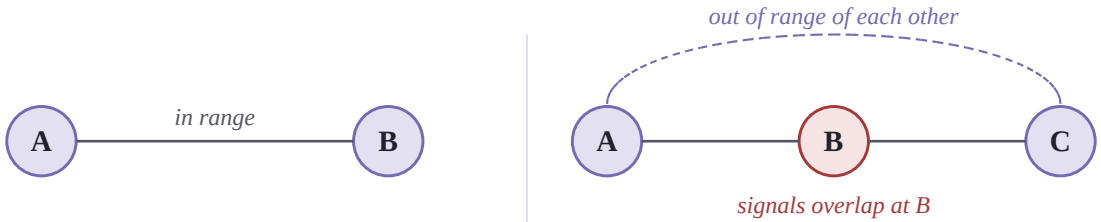
Ref	Requirement	How it is met	Section
R1	networkx graph of radios and links; radios within 500 m can talk	Edge for every pair at most 500 m apart; range set by `--range`	4.1
R2	Distance-2 colouring; colours are exclusive slots in a recurring frame	Proper colouring of G^2, equivalent to distance-2 colouring; colour k is slot k	4.2, 4.3
R3	Direct link interference: different slots	Every link of G is an edge of G^2	4.2, 5.5

Ref	Requirement	How it is met	Section
R4	Hidden terminal: different slots	Radios with a common neighbour are joined in G^2	4.2, 5.5
R5	Spatial reuse beyond two hops	Radios not joined in G^2 may share a slot; 5 of 9 slots are shared on the grid	5.1
R6	Heuristics to reduce the number of slots	DSATUR and largest-first, shorter frame kept; lower bound reported	4.3 to 4.5, 5.4
R7	CLI taking 16 node coordinates as a JSON string	`src/cli.py` takes the JSON string, or a file with `--file`	5.3
R8	Slot × Node boolean matrix and node-to-slot mapping	Printed in the layout of the sample output	5.3
R9	Graded on collision avoidance and heuristic effectiveness	Independent validator; benchmark on 1,500 random layouts	4.6, 5.4
R10	EMANE on one Linux machine or in one Docker container	EMANE 1.5.3 built from source in one container	6.1
R11	TDMA radio model XML with 1 ms slots, frames and frequencies	1 ms slots, one frame per multiframe, 2.4 GHz, 1 MHz bandwidth	6.3, 6.4
R12	Bridge to an XML or ProtoBuf schedule event	`bridge.py` writes the schedule XML; `emaneevent-tdmaschedule` sends one protobuf event per NEM	6.3
R13	EMANE drops or permits packets according to the schedule	Delivery on all 84 in-range links, compared with a control schedule	7
R14	Documentation of design, reasoning and final values	Design: 4, 6. Reasoning: 4.7, 5.2, 6.2. Final values: 5, 7	4 to 7
R15	Git repository with source code, PDF and PPT	Public repository, linked on the cover and in Appendix A	App. A

3. System Model and Assumptions

3.1 Interference Model

The task defines two situations that a schedule must prevent (Figure 1). In **direct interference**, radios A and B are in range of each other, and if both transmit in the same slot neither can receive the other. In the **hidden-terminal** case, A and C are out of range of each other but both in range of B; neither can sense the other, so if they share a slot their signals overlap at B. Radios more than two hops apart have no receiver in common and may therefore share a slot, which is what allows spatial reuse.



(a) Direct interference (1 hop)
A and B must use different slots

(b) Hidden terminal (2 hops)
A and C must use different slots

Figure 1. The two interference cases the schedule must prevent.

3.2 Assumptions

Table 3. Modelling assumptions.

Item	Assumption	Reason
Coordinates	4 × 4 grid, 300 m spacing, numbered row by row from Node_01 (0, 0) to Node_16 (900, 900) (`data/grid16.json`)	The task gives only Node_01, Node_02 and Node_16; a regular grid is the simplest consistent layout
Radio range	Linked if at most 500 m apart	Stated in the task; configurable with `--range`
Slots per radio	One transmit slot per radio per frame	The task asks for a node-to-slot mapping; traffic load is not modelled
Mobility	Static positions	The task gives static coordinates
Synchronisation	All radios share the same slot boundaries	Implied by the synchronised frame; Section 7.3 shows the effect of emulator timing

4. Part 1: Design Process

Part 1 turns coordinates into a slot assignment in five steps: build the radio graph, derive the conflict graph, colour it, compare the result with a lower bound, and check the answer again by an independent method.

4.1 Radio Graph

Each radio is a node of an undirected networkx graph G [1]. For every pair of radios the Euclidean distance is computed, and an edge is added when it is at most the range R (500 m by default), so an edge means that two radios can hear each other. A distance of exactly 500 m counts as in range. With 16 radios there are only 120 pairs, so checking every pair is the simplest correct method. On the grid, G has **42 edges**: 24 horizontal and vertical links of 300 m and 18 diagonal links of about 424 m.

4.2 Conflict Graph

Both interference cases involve radios at most two hops apart in G. Joining every such pair gives the **conflict graph** $H = G^2$, built with `networkx.power(G, 2)`. A distance-2 colouring of G is exactly a proper colouring of H, so the task becomes: colour H with as few colours as possible, where each colour is one time slot. On the grid, H has **90 edges**.

4.3 Greedy Colouring Heuristics

Minimum distance-2 colouring is NP-hard [4], so the tool uses greedy colouring: visit the nodes one at a time and give each the lowest slot that none of its neighbours in H holds. Greedy colouring **can never put two conflicting radios in the same slot**; the visiting order only changes how many slots are needed, and choosing that order is the heuristic. **Largest-first** [2] visits nodes by degree in H, highest first, in a fixed order. **DSATUR** [3] always picks next the uncoloured node whose neighbours hold the most distinct slots, breaking ties by degree, so the order adapts as slots are handed out (Algorithm 1).

Algorithm 1. DSATUR slot assignment on the conflict graph *H*.

```
U <- all nodes of H                                  # uncoloured nodes
while U is not empty:
    sat(v) <- number of distinct slots held by neighbours of v in H
    v <- node in U with the largest sat(v); ties: largest degree in H
    slot[v] <- smallest slot index not held by any neighbour of v in H
    remove v from U
```

Both routines are written by hand in `graph_engine.py` (`dsatur`, `largest_first`) so that every step can be explained, and the tests confirm that they give exactly the same colourings as networkx's `greedy_color`.

4.4 Selection and Normalisation

Both heuristics run on every input and the shorter schedule is kept, with DSATUR preferred on a tie. Slots are then relabelled in order of first use, so the first node always gets slot 0 and the same input always gives the same report, tests and EMANE schedule file.

4.5 Lower Bound

Any radio *v* and its neighbours are pairwise at most two hops apart (through *v*), so they form a clique in *H* and need different slots. The radio with the most neighbours, d_{max}, therefore forces at least $\boldsymbol{d_{max} + 1}$ **slots**. When a schedule reaches this bound it is provably optimal, and the report says so.

4.6 Independent Validation

Before printing, `validator.py` re-checks the schedule from the raw coordinates rather than from *H*: every two radios that share a slot must be out of range of each other and have no common neighbour. Because the check does not reuse the conflict graph, **an error in building that graph cannot hide a collision**. If any pair fails, the tool prints it and exits with status 1.

4.7 Design Decisions

Table 4. Design decisions in Part 1.

Decision	Alternative considered	Reason
Colour the square of the radio graph	A dedicated distance-2 colouring routine	Same schedules; standard colouring heuristics then apply directly
networkx for the graph; colouring by hand	networkx's `greedy_color` for the colouring	The task names networkx and asks for the algorithm to be written; checked against networkx in tests
Run two heuristics, keep the better	DSATUR only	The second run is cheap and sometimes shorter (5.4)
Report a lower bound	Frame length only	Shows whether the result is provably optimal
Validate from raw coordinates	Check the conflict graph's edges	A check that reuses the conflict graph would pass even if the graph were wrong

5. Part 1: Final Values

5.1 Schedule for the 16-Node Grid

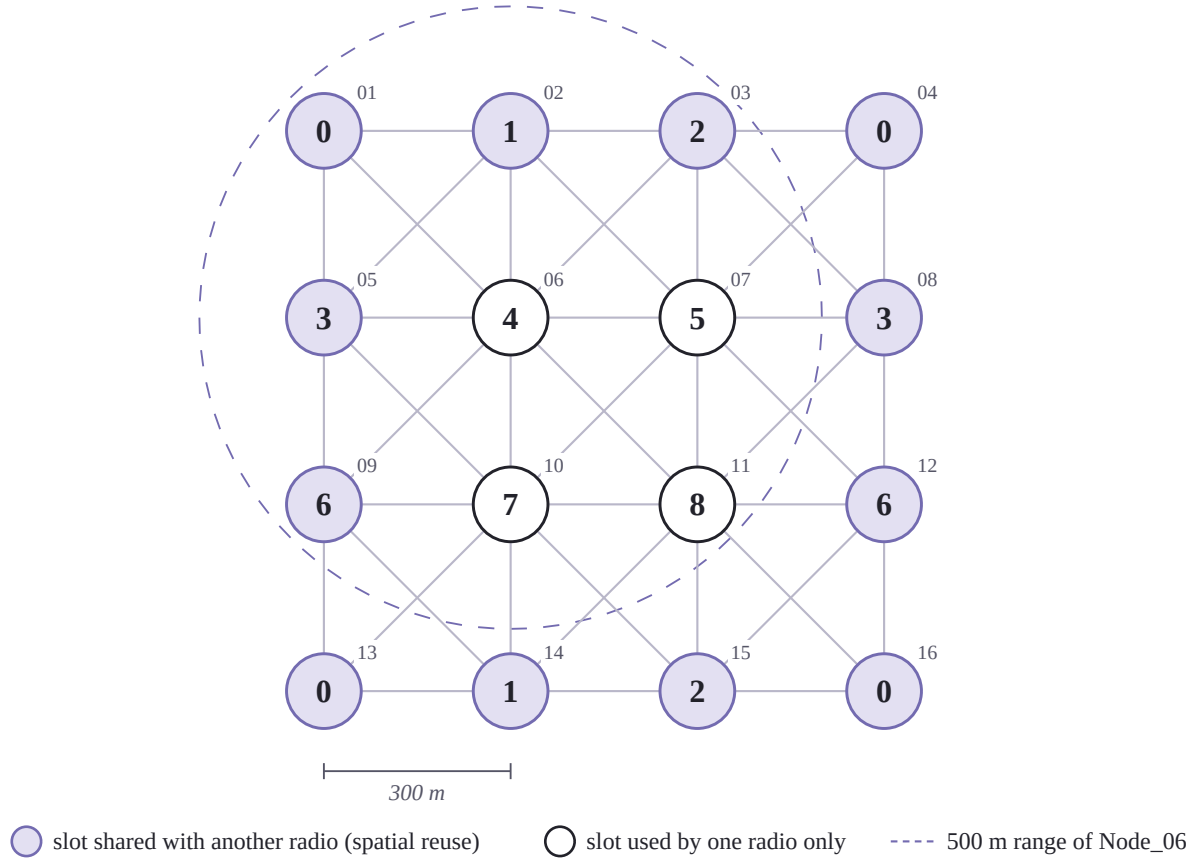


Figure 2. The 16-node grid and its 42 links. Each circle shows the radio's slot; the small label is the node number. The dashed circle is the 500 m range of Node_06.

Figure 2 shows the result: a **9-slot frame for 16 radios** from both heuristics, equal to the lower bound, with no collisions found by the validator. The four corner radios share slot 0, and slots 1, 2, 3 and 6 are each shared by two radios on opposite edges, three hops apart. The four interior radios are within two hops of every other radio, so each needs its own slot: 1 + 4 + 4 = 9, with 12 of the 16 radios transmitting in a shared slot.

5.2 Why 9 Slots and Not the 5 in the Sample Output

With a 500 m range each radio reaches its horizontal and vertical neighbours (300 m) and its diagonal neighbours (about 424 m). An interior radio such as Node_06 has eight neighbours, and by Section 4.5 those nine radios need nine different slots. **No valid distance-2 schedule for this grid can be shorter than 9 slots.**

Applied to this grid, the sample's 5-slot assignment separates direct neighbours but not hidden terminals: Node_01 and Node_03 both hold slot 0 while both reach Node_02, and in total it places 21 pairs of two-hop neighbours in the same slot, so the validator rejects it. I therefore read the sample as an illustration of the report format, not as a target. Two results support this: with a 350 m range (no diagonal links) the tool returns exactly 5 slots for the same grid, and in EMANE the 5-slot assignment loses most of its traffic to interference (Section 7.2).

5.3 Command-Line Output

The CLI takes the coordinates as a JSON string in the task's format, or from a file with `--file`. Listing 1 is its complete output for the grid; in the matrix, a 1 means the radio transmits in that slot.

Listing 1. CLI output for the 16-node grid.

```
$ python src/cli.py '{"Node_01":[0.0,0.0],"Node_02":[300.0,0.0],...,"Node_16":[900.0,900.0]}'
============================================================
 TDMA TOPOLOGY OPTIMIZATION REPORT
============================================================
Total Nodes Processed  : 16
Configured Radio Range : 500.0 meters
Optimized Frame Length : 9 unique timeslots (Lower is better)
Heuristics Compared    : DSATUR = 9, largest_first = 9 (using DSATUR)
Slot Lower Bound       : 9 -> schedule is optimal
------------------------------------------------------------
NODE -> SLOT ASSIGNMENTS:
 Node_01: Slot 0
 Node_02: Slot 1
 Node_03: Slot 2
 Node_04: Slot 0
 Node_05: Slot 3
 Node_06: Slot 4
 Node_07: Slot 5
 Node_08: Slot 3
 Node_09: Slot 6
 Node_10: Slot 7
 Node_11: Slot 8
 Node_12: Slot 6
 Node_13: Slot 0
 Node_14: Slot 1
 Node_15: Slot 2
 Node_16: Slot 0

STRUCTURAL TDMA SCHEDULE MATRIX (Slot x Node Boolean Matrix):
Slot \ Node | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 10 | 11 | 12 | 13 | 14 | 15 | 16
-----------------------------------------------------------------------------------------------
Slot 00     |  1 |  0 |  0 |  1 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  1 |  0 |  0 |  1
Slot 01     |  0 |  1 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  1 |  0 |  0
Slot 02     |  0 |  0 |  1 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  1 |  0
Slot 03     |  0 |  0 |  0 |  0 |  1 |  0 |  0 |  1 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0
Slot 04     |  0 |  0 |  0 |  0 |  0 |  1 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0
Slot 05     |  0 |  0 |  0 |  0 |  0 |  0 |  1 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0
Slot 06     |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  1 |  0 |  0 |  1 |  0 |  0 |  0 |  0
Slot 07     |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  1 |  0 |  0 |  0 |  0 |  0 |  0
Slot 08     |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  0 |  1 |  0 |  0 |  0 |  0 |  0
-----------------------------------------------------------------------------------------------
Execution finalized cleanly. Schedule verified conflict-free.
============================================================
```

5.4 Comparison of the Heuristics

To compare the heuristics beyond the grid, both were run on 500 random layouts per network size, with radios placed uniformly in a 2 km × 2 km area and a 500 m range (Table 5).

Table 5. DSATUR and largest-first on 500 random layouts per network size.

Radios	DSATUR shorter	Largest-first shorter	Same length	Best result equals lower bound
16	3	0	497	481 of 500
30	25	1	474	439 of 500
60	142	17	341	349 of 500

DSATUR is the stronger heuristic and its lead grows with network size, but largest-first still won 18 times, which is why both are run. Where the best result missed the lower bound it was at most 3

slots above it; since the bound is not always attainable, some of those schedules may still be optimal.

5.5 Test Suite

The pytest suite has 98 tests. They cover a network with no links (all radios in slot 0), a fully connected network (one slot each), the A-B-C hidden-terminal line (three slots), a 9-radio line (3 slots, reuse at three hops), two radios exactly 500 m apart, the grid with both heuristics (9 slots, no collisions) and 20 random layouts per heuristic. The validator must flag a direct and a hidden-terminal collision and reject the task's 5-slot sample. Further tests check DSATUR on textbook graphs, compare the hand-written colouring with networkx's, exercise the CLI including malformed input, and verify the transmitters the EMANE bridge lists in each slot.

6. Part 2: Design Process

Part 2 shows in an emulator that packets are delivered or lost according to the Part 1 schedule. The EMANE setup was kept as close as possible to the Part 1 model, so that any difference in delivery can be traced back to the schedule.

6.1 Emulation Architecture

Everything runs in one Docker container (Figure 3), started with `--privileged` because the demo creates network namespaces and TUN devices. EMANE 1.5.3 is compiled from source in the image, so the same Dockerfile builds on x86 and ARM. Each radio runs in its own namespace as one EMANE instance with one NEM, using EMANE's native TDMA radio model and a virtual interface `emane0` (radio *N* is 10.100.0.*N*). A Linux bridge, `emanebr`, carries EMANE's over-the-air and event channels, and both the schedule and the pathloss reach the radios as events over it.

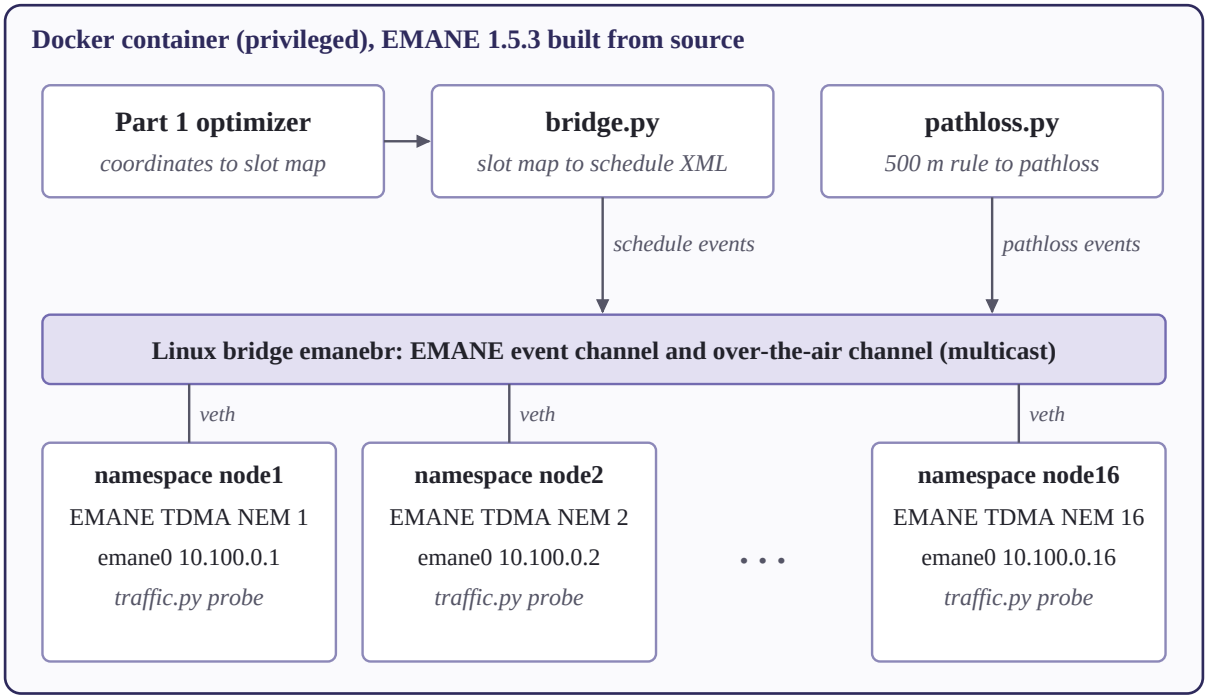


Figure 3. Emulation layout: one container, one network namespace and one EMANE TDMA radio per node.

6.2 Mapping the Part 1 Model onto EMANE

Table 6. Correspondence between the Part 1 model and EMANE.

Part 1 concept	EMANE equivalent
Node, in input order	NEM id 1 to *N* (Node_01 is NEM 1)
Slot (colour) of a node	NEM id listed in that slot's element, with a `tx` child
Frame length	`slots` attribute of the schedule structure; one frame per multiframe
Silent in all other slots	EMANE default: unassigned slots are receive slots
Within 500 m: linked	Pathloss event with the free-space loss at 2.4 GHz
Beyond 500 m: no interaction	Pathloss event of 300 dB, far below receiver sensitivity

EMANE has no radio-range parameter; what a receiver hears depends only on pathloss. `pathloss.py` therefore applies the Part 1 rule, and the physical layer uses EMANE's precomputed propagation model, so these events are the only source of pathloss.

6.3 Integration Bridge

`bridge.py` imports the Part 1 engine, computes the schedule and writes it as EMANE TDMA schedule XML (Listing 2): one frame of nine slots, each 1000 microseconds (1 ms) long, with each `slot` element listing its transmitters.

Listing 2. Start of the generated EMANE schedule file for the grid.

```
<emane-tdma-schedule>
  <structure frames="1" slots="9" slotoverhead="0" slotduration="1000" bandwidth="1M" />
  <multiframe frequency="2.4G" power="20" datarate="11M" class="0">
    <frame index="0">
      <slot index="0" nodes="1,4,13,16">
        <tx />
      </slot>
      <slot index="1" nodes="2,14">
        <tx />
      </slot>
      ...
```

EMANE's `emaneevent-tdmaschedule schedule.xml -i emanebr` turns the file into **one protobuf schedule event per NEM**, each carrying only that NEM's slots. The demo then reads each radio's `scheduler.scheduleAcceptFull` counter and `ScheduleInfoTable` to confirm the schedule was accepted as written.

6.4 Radio Configuration

Table 7. Radio settings.

Setting	Value	Reason
Slot duration	1 ms; 10 ms for comparison	1 ms is the task's value; 10 ms separates the schedule's effect from emulator timing
Frames; slot overhead	1; 0	One repeating frame is all the schedule needs
Frequency; bandwidth	2.4 GHz; 1 MHz	Single shared channel, as in the task
Data rate	11 Mbit/s	A 1 ms slot carries 1,375 bytes; the default curve needs SINR above 1 dB, so overlapping signals of similar strength are lost and collisions are visible

Setting	Value	Reason
Transmit power; antenna gain	20 dBm; 0 dBi	SNR far above the 8 dB needed for full reception (Table 8)
Noise mode; noise figure	all; 4 dB	Every in-band transmission counts as interference
Reception curve	EMANE default	`tdmabasemodelpcr.xml`, no custom tuning

Table 8. Free-space link budget at 2.4 GHz (calculated from the settings, not measured).

Link	Distance	Path loss	Received power	SNR
Horizontal or vertical neighbour	300 m	89.6 dB	–69.6 dBm	40.4 dB
Diagonal neighbour	424 m	92.6 dB	–72.6 dBm	37.4 dB
Out of range	600 m or more	300 dB (set)	–280 dBm	below sensitivity

Noise floor: –174 dBm/Hz + 10 $\log_{10}$(1 MHz) + 4 dB = –110 dBm. Path loss = 20 $\log_{10}(d)$ + 20 $\log_{10}(f)$ – 147.55 dB (d in m, f in Hz).

Every in-range link has an SNR above 35 dB, while two overlapping in-range signals differ by at most 3 dB, far below 8 dB. Collisions therefore show up clearly as SINR drops.

6.5 Test Method

All 16 radios broadcast at once, each sending 300 UDP packets of 64 bytes at 50 packets per second through `emane0` (`traffic.py`). `report.py` computes delivery on the 84 directed in-range links (25,200 packets expected), counts packets heard beyond 500 m, and sums EMANE's MAC drop table: *SINR* means loss to interference, while *Slot Error* and *Long* are timing reasons (the packet arrived outside its slot, or too late to finish within it). The optimizer's 9-slot schedule is compared with a control that separates only direct neighbours, the sample's 5-slot assignment (`emane/distance1_schedule.json`), at both slot durations. The demo was run three times.

7. Part 2: Final Values

The results meet the Part 2 deliverable: EMANE ran the Part 1 schedule as generated, lost **no packets to interference** with it, and dropped packets where the control schedule allowed hidden-terminal collisions. Table 9 gives the recorded run (`emane/out`) and Figure 4 the per-radio delivery with 10 ms slots.

Table 9. Recorded EMANE run: 84 directed in-range links, 300 packets per radio. Timing drops are the Slot Error and Long reasons.

Schedule	Slot	Packets delivered	Links with loss	SINR drops (bytes)	Timing drops (bytes)	Out-of-range packets
Optimizer, 9 slots	1 ms	21,009 of 25,200 (83.4%)	84 of 84	**0**	446,276	0
Distance-1 control, 5 slots	1 ms	16,423 of 25,200 (65.2%)	84 of 84	274,992	360,596	0
Optimizer, 9 slots	10 ms	**25,168 of 25,200 (99.9%)**	8 of 84	**0**	3,392	0
Distance-1 control, 5 slots	10 ms	8,700 of 25,200 (34.5%)	55 of 84	763,194	0	0

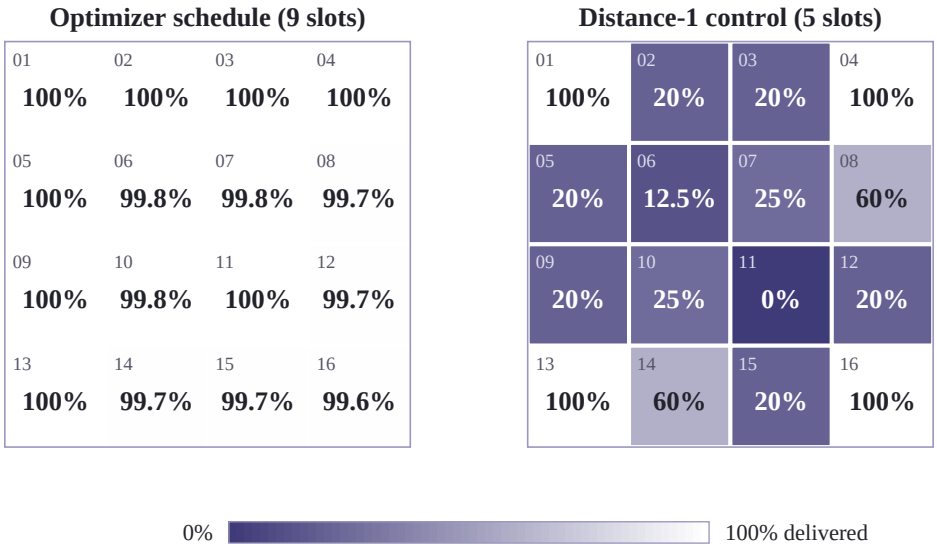


Figure 4. Share of its neighbours' packets received by each radio with 10 ms slots, laid out as in Figure 2. Darker cells mean more loss.

7.1 Schedule Enforcement and Collision Avoidance

Every radio accepted the schedule event, and each ScheduleInfoTable showed one transmit slot and receive in all others, exactly as generated by the bridge. With the optimizer's schedule EMANE recorded **no SINR drops at either slot duration in any of the three runs**. With 10 ms slots, 99.9% of packets were delivered in the recorded run and 100% in the other two; the small loss appears only under timing reasons.

7.2 Cost of Ignoring Hidden Terminals

The 5-slot control delivered only **34.5% with 10 ms slots** in all three runs, every lost byte being an SINR drop, with loss on 55 of the 84 links. The pattern in Figure 4 follows the slot assignment: Node_11 received nothing because each of its eight neighbours shares a slot with another of them, while the corner radios, whose three neighbours hold different slots, received everything. At 1 ms the control reached 65.2% only because of **lighter channel use, not a better schedule**: a 5 ms frame repeats 200 times per second, so at 50 packets per second a radio uses about one slot in four

and slot-sharing radios often do not transmit in the same frame. With 10 ms slots every slot carries traffic and every shared slot collides.

7.3 Range Model and Timing at 1 ms

No packet was heard between radios more than 500 m apart, so the pathloss events reproduced the range rule. With the optimizer's schedule at 1 ms, 83.4% of packets were delivered in the recorded run, and all of the loss had timing reasons. Most transmissions left 0.25 to 0.5 ms into their slot, and some arrived after it ended, where the model discards them. This is **scheduling jitter from 16 EMANE processes on a 4-CPU virtual machine**, not a property of the schedule, and it varied between runs (99.4%, 88.1% and 83.4%; logs in `emane/out/repeats`). The 10 ms results, with the same schedule and radios, show what the schedule achieves once timing is not the limit.

8. Limitations and Future Work

Table 10. Limitations and possible extensions.

Limitation	Effect and possible extension
Assumed coordinates	Only three coordinates are given; any other layout needs only a new input file
Disc interference model	Distant transmitters are ignored, though real receivers see their summed interference; an SINR-based conflict model would capture this at higher cost
One slot per radio	Busy relays get the same share as edge radios; extra slots could be added where the frame has room
Static, centralised schedule	Moving radios need a recomputed schedule; EMANE accepts new schedule events, so the bridge can simply be re-run
Heuristic, not exact	Optimality is proven only at the lower bound; integer programming could confirm or improve results at this network size
Emulation timing	1 ms slots lose packets to jitter in a 4-CPU VM; bare-metal Linux with more cores should reduce this (not yet tested)

9. Conclusion

The optimizer reduces the task's two interference rules to colouring the square of the radio graph. Greedy colouring with two orders keeps the frame short, the lower bound shows how good the result is, and the validator re-checks it from the raw coordinates; for the 16-node grid the result is a **provably optimal 9-slot schedule**. EMANE confirms it on a running network: no packet was lost to interference with the optimizer's schedule, while a schedule that ignored hidden terminals lost most of its traffic. The remaining loss at 1 ms comes from emulator timing, listed with the other next steps in Section 8.

Appendix A. Reproducing the Results

The source code, this document and the presentation are in a public repository:

github.com/srikarreddyram/tdma-schedule-optimizer

Part 1 needs Python 3.10 or newer and Part 2 needs Docker; the first build compiles EMANE and is slow. Packets per radio and slot durations can be changed with `PACKETS` (default 300) and `SLOT_DURATIONS_US` (default `"1000 10000"`) via `docker run -e`.

Listing 3. Commands to fetch the code and run the tool, tests and emulation.

```
git clone https://github.com/srikarreddyram/tdma-schedule-optimizer.git
cd tdma-schedule-optimizer
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python src/cli.py --file data/grid16.json
.venv/bin/python src/cli.py --file data/grid16.json --range 350      # 5-slot case
.venv/bin/python -m pytest
docker build -t tdma-emane -f emane/Dockerfile .
docker run --rm --privileged -v "$PWD/emane/out:/work/emane/out" tdma-emane
```

Table 11. Repository layout.

Path	Contents
`src/`	`graph_engine.py` (graphs, colouring, lower bound), `validator.py`, `formatter.py` (report), `cli.py`
`tests/`, `pytest.ini`	pytest suite (98 tests)
`data/grid16.json`	The 16-node test set
`emane/`	`Dockerfile`, `xml/` (radio configuration), `bridge.py`, `pathloss.py`, `traffic.py`, `report.py`, `run_demo.sh`, `out/` (results)
`docs/`	This document and its build script
`presentation.pptx`, `README.md`	Slides; overview and usage

References

[1] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conference (SciPy 2008)*, Pasadena, CA, 2008, pp. 11–15.

[2] D. J. A. Welsh and M. B. Powell, "An upper bound for the chromatic number of a graph and its application to timetabling problems," *The Computer Journal*, vol. 10, no. 1, pp. 85–86, 1967.

[3] D. Brélaz, "New methods to color the vertices of a graph," *Communications of the ACM*, vol. 22, no. 4, pp. 251–256, 1979.

[4] S. T. McCormick, "Optimal approximation of sparse Hessians and its equivalence to a graph coloring problem," *Mathematical Programming*, vol. 26, no. 2, pp. 153–171, 1983.

[5] S. Ramanathan and E. L. Lloyd, "Scheduling algorithms for multihop radio networks," *IEEE/ACM Transactions on Networking*, vol. 1, no. 2, pp. 166–177, 1993.

[6] EMANE TDMA Radio Model documentation, https://emane.io/tdma-radio-model.